

Beyond machines how we could analyze algorithms?

- ❖ IDEA - **Asymptotic Analysis** – a method of describing limiting behavior
 - ❖ Ignore machine dependent constants
 - ❖ Look at **growth of running time** $T(n)$ when $n \rightarrow \infty$

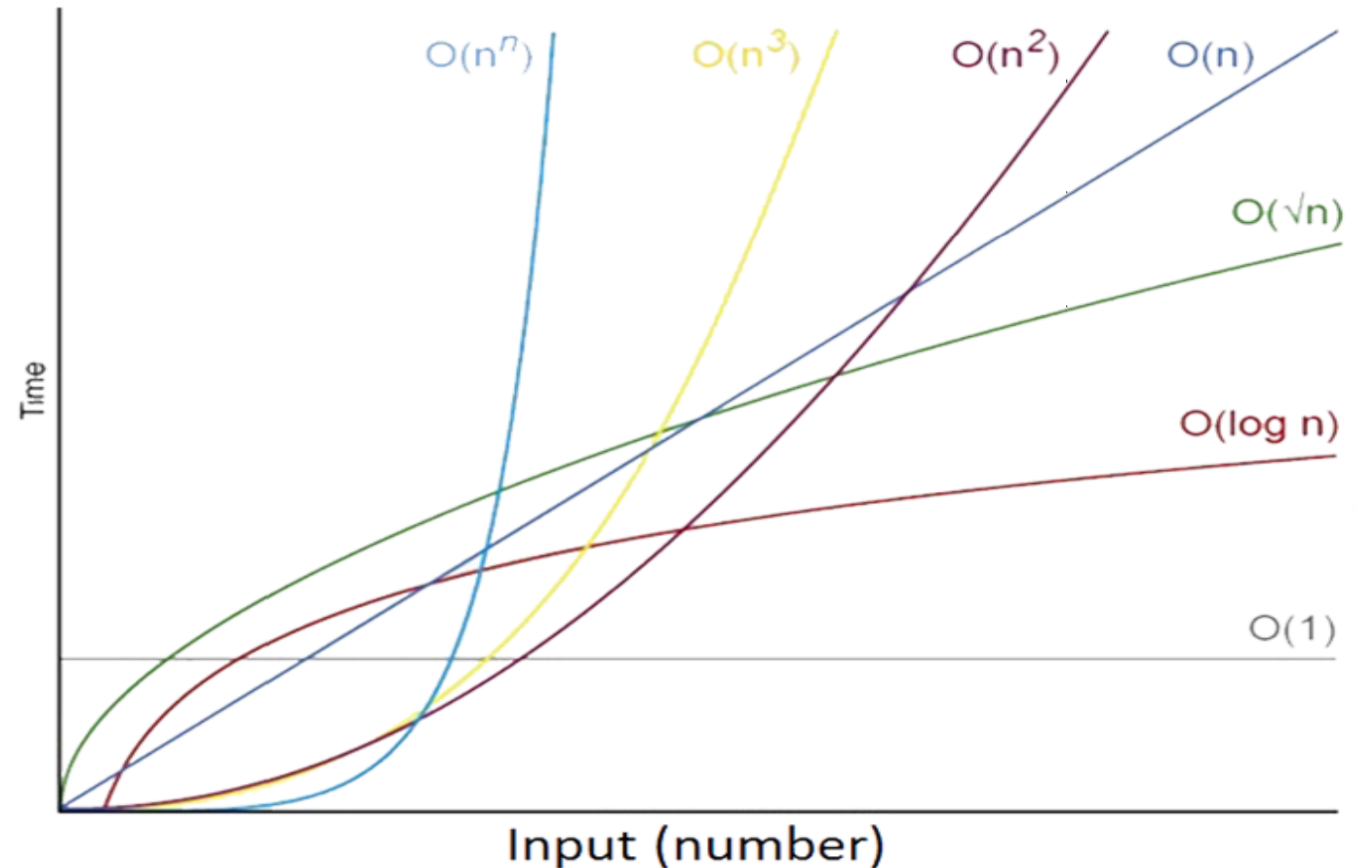
Order of Growth

- Performing step count calculation for large algorithms is a time-consuming task
- A natural way is to upper bound the time complexity instead of finding the exact step count
- **Order of Growth or Rate of Growth** of an algorithm gives a simple characterization of the algorithm's efficiency by identifying relatively significant term in the step count
- For an algorithm with a step count $2n^2 + 3n + 1$, the order of growth depends on $2n^2$ for large n . Other terms are relatively insignificant as n increases
- **Asymptotic analysis** is a technique that focuses analysis on the 'significant term'

Comparison of Time Functions

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$$

$\log n$	n	n^2	2^n
0	1	1	2
1	2	4	4
2	4	16	16
3	8	64	256
3.16	9	81	512
3.22	10	100	1024



Orders of magnitude

- When comparing $t(n)$ across problems, focus on orders of magnitude
- Ignore constants
- $f(n) = n^3$ eventually grows faster than $g(n) = 5000 n^2$
- For small values of n , $f(n)$ is smaller than $g(n)$, At $n = 5000$, $f(n)$ overtakes $g(n)$
- **What happens in the limit, as n increases :
Asymptotic complexity**

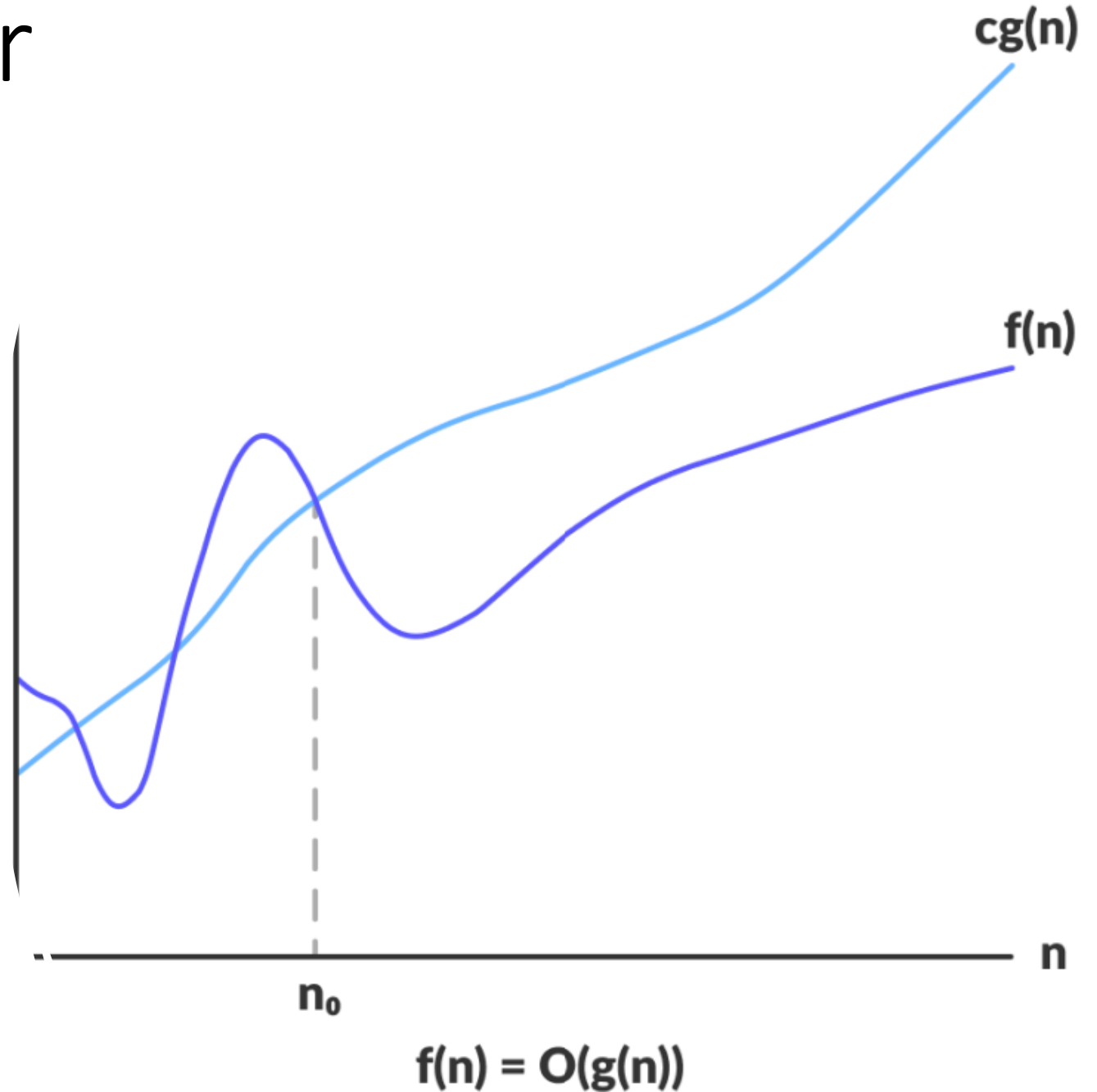
Asymptotic analysis

- Commonly used asymptotic notations are
 1. **Big-oh Notation (O)** - To express an upper bound on the time complexity as a function of the input size
 2. **Omega (Ω)** - To express a lower bound on the time complexity as a function of the input size
 3. **Theta (Θ)** - To express the tight bound on the time complexity as a function of the input size

Asymptotic upper bounds

Big-oh notation

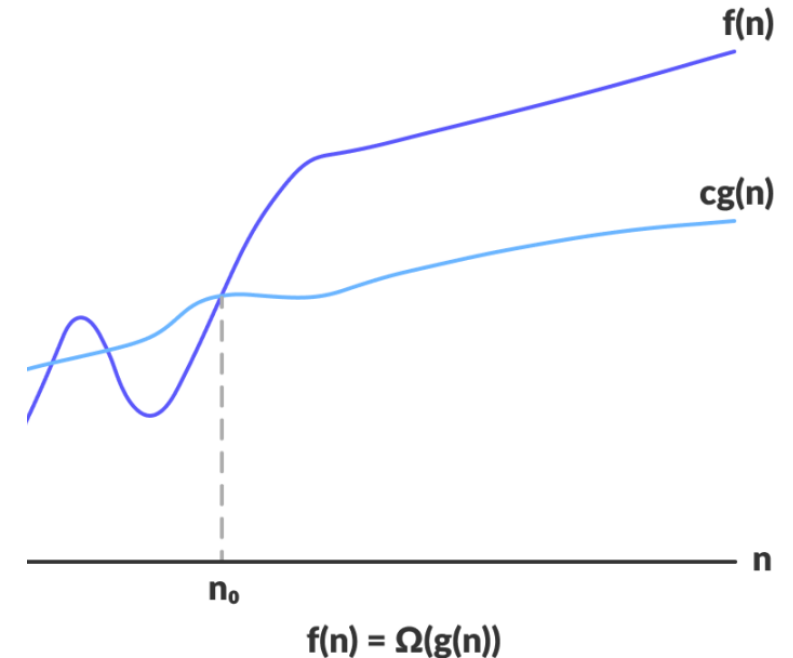
- The function $f(n) = O(g(n))$ if and only if there exist positive constants c, n_0 such that $f(n) \leq cg(n), \forall n \geq n_0$.
- Big-oh can be used to denote all upper bounds on the time complexity of an algorithm.
- Big-oh also captures the worst-case analysis of an algorithm.



Asymptotic lower bounds

Omega notation

- The function $f(n) = \Omega(g(n))$ if and only if there exist positive constants c, n_0 such that $f(n) \geq c \cdot g(n), \forall n \geq n_0$.
- Omega can be used to denote all lower bounds of an algorithm.
- Omega notation also denotes the best-case analysis of an algorithm.
 - $3n + 2 \geq n, \forall n \geq 1 \Rightarrow 3n + 2 = \Omega(n)$
 - $10n^2 + 4n + 2 = \Omega(n^2), c=1, \forall n \geq 1$
 - $2n^2 + n \log n + 1 \geq 2 \cdot n^2, c = 2, \forall n \geq 1$
 - $6 \cdot 2^n + n^2 = \Omega(2^n) = \Omega(n^2) = \Omega(n) = \Omega(1)$
 - $6 \cdot 2^n + n^2 \geq 2^n, c = 1 \forall n \geq 1$
 - $\Omega(2^n)$ is a tight lower bound, while all others are **loose lower bounds**.
- $3n^2 + 2 \neq \Omega(n^3)$. Reason: There does not exist a positive constant c such that $3n^2 + 2 \geq c \cdot n^3$ for every $n \geq n_0$ as we cannot bound n by a constant.
- $3 \cdot 2^n \neq \Omega(n!)$
- $5 \neq \Omega(n)$



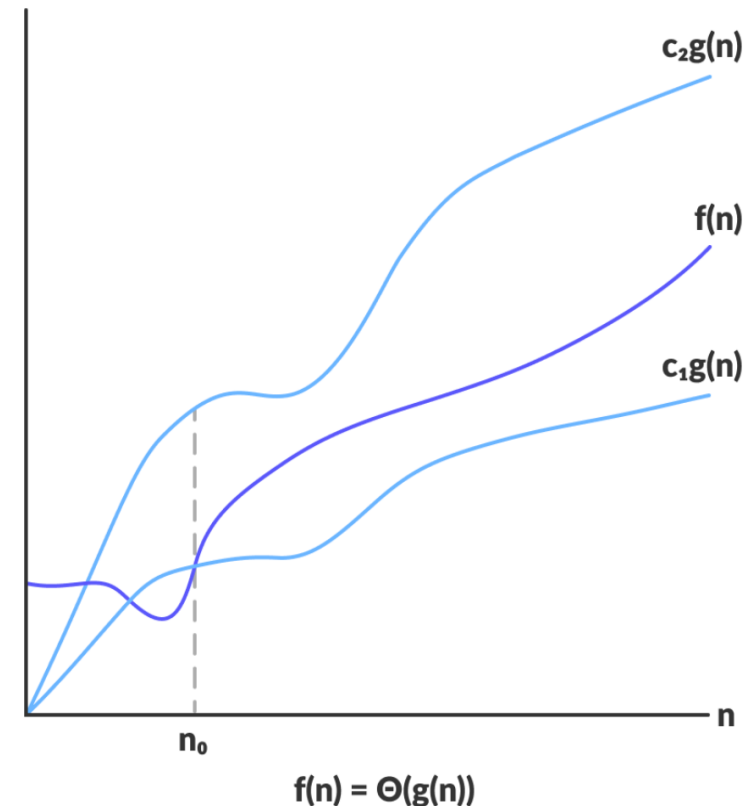
Asymptotic tight bound

Theta notation

- The function $f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n), \forall n \geq n_0$. Theta can be used to denote tight bounds of an algorithm.
- i.e., $g(n)$ is a lower bound as well as an upper bound for $f(n)$. Note that $f(n) = \Theta(g(n))$ if and only if $f(n) = \Omega(g(n))$ and $f(n) = O(g(n))$.

Examples:

- $3n + 10^{10}, 3n \leq 3n + 10^{10} \leq 4n, \forall n \geq 10^{10} \Rightarrow 3n + 10^{10} = \Theta(n)$
- $10n^2 + 4n + 2 = \Theta(n^2), 10n^2 \leq 10n^2 + 4n + 2 \leq 20n^2, \forall n \geq 1, = 20$
- $6(2^n) + n^2 = \Theta(2^n), 6(2^n) \leq 6(2^n) + n^2 \leq 12(2^n), \forall n \geq 1, c_1 = 6,$
 - $3n + 2 \neq \Theta(1).$ Reason: $3n + 2 \neq O(1)$
 - $3n + 3 \neq \Theta(2).$ Reason: $3n + 3 \neq \Omega(n^2)$
 - $n^2 \neq \Theta(2n).$ Reason: $n^2 \neq \Omega(2n)$



Tight bounds

- * $n(n-1)/2$ is $\Theta(n^2)$

- * Upper bound

$$n(n-1)/2 = n^2/2 - n/2 \leq n^2/2, \text{ for } n \geq 0$$

- * Lower bound

$$n(n-1)/2 = n^2/2 - n/2 \geq n^2/2 - (n/2 \times n/2) \geq n^2/4, \\ \text{for } n \geq 2$$

- * Choose $n_0 = \max(0,2) = 2$, $c_1 = 1/2$ and $c_2 = 1/4$

Summary

- * $f(n) = O(g(n))$ means $g(n)$ is an upper bound for $f(n)$
 - * Useful to describe limit of worst case running time for an algorithm
- * $f(n) = \Omega(g(n))$ means $g(n)$ is a lower bound for $f(n)$
 - * Typically used for classes of problems, not individual algorithms
- * $f(n) = \Theta(g(n))$: matching upper and lower bounds
 - * Best possible algorithm has been found

Property:

- If $f_1(n)$ is $O(g_1(n))$, $f_2(n)$ is $O(g_2(n))$, then $f_1(n) + f_2(n)$ is $O(\max(g_1(n), g_2(n)))$

Why is this important?

Algorithm has two phases

- Phase A takes time $O(g_A(n))$
- Phase B takes time $O(g_B(n))$
- Algorithm as a whole takes time $\max(O(g_A(n)), O(g_B(n)))$
- For an algorithm with many phases, least efficient phase is an upper bound for the whole algorithm

